

AI Prompt Log — CSE 4334/5334 Project 1

AI tool: Claude (Anthropic), web chat interface **Model version:** Claude Opus 5 **Log date:** 2026-09-24 **Course:** CSE 4334/5334 Data Mining — Fall 2026 **Project:** Project 1 — Data Preprocessing & Pipeline Construction **Purpose of this log:** Section 5 of the project specification requires that every prompt submitted to an AI tool be logged with the prompt text, the model version, the raw output, and the team's code modifications.

Note for the report: the "Team code modifications" fields below are left blank for the team to fill in. This log records the AI side of the exchange only.

Turn 1

Prompt:

how to load csv data into python

Raw AI output:

The two standard ways are **pandas** (for analysis) and Python's built-in `csv` module (no dependencies).

1. pandas (most common)

```
import pandas as pd

df = pd.read_csv("data.csv")
print(df.head())      # first 5 rows
print(df.info())      # column types and null counts
```

Useful `read_csv` options:

```
df = pd.read_csv(
    "data.csv",
    sep="," ,                # delimiter; use "\t" for TSV
    header=0,                # row with column names; None if there isn't one
    usecols=["id", "price"], # load only some columns
    dtype={"id": str},       # force column types
    parse_dates=["date"],    # convert to datetime
```

```

na_values=["NA", ""],      # extra strings to treat as missing
nrows=1000,                # read only the first N rows
encoding="utf-8",          # try "latin-1" if you get a UnicodeDecodeError
)

```

For files too large for memory, read in chunks:

```

for chunk in pd.read_csv("big.csv", chunksize=100_000):
    process(chunk)

```

2. Built-in `csv` module

```

import csv

with open("data.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)    # each row is a dict keyed by header
    for row in reader:
        print(row["name"], row["price"])

```

Use `csv.reader(f)` instead if you want each row as a plain list. Note that all values come back as strings, so you convert types yourself. The `newline=""` argument is recommended by the Python docs so line endings inside quoted fields are handled correctly.

3. NumPy (numeric-only data)

```

import numpy as np
arr = np.loadtxt("data.csv", delimiter=",", skiprows=1)

```

Which to use

- **pandas**: data analysis, ML preprocessing, mixed column types.
- **csv** : lightweight scripts, streaming row by row, no install needed.
- **NumPy**: purely numeric matrices.

References: [pandas read_csv docs](#), [Python csv module docs](#), [NumPy.loadtxt docs](#).

Team code modifications:

(to be filled in)

Turn 2

Prompt:

I'm going to be using pandas and scikit learn

Raw AI output:

Here's the typical workflow for going from a CSV to a scikit-learn model.

1. Load and inspect

```
import pandas as pd

df = pd.read_csv("data.csv")
print(df.shape)
print(df.dtypes)
print(df.isna().sum())    # missing values per column
```

2. Separate features and target

```
X = df.drop(columns=["target"])
y = df["target"]
```

scikit-learn accepts DataFrames directly, so you don't need to convert to NumPy.

3. Split before any preprocessing

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y # stratify for classification
)
```

Splitting first matters: if you fit a scaler or imputer on the full dataset, test-set information leaks into training. The scikit-learn docs call this out explicitly in their "Common pitfalls" guide.

4. Preprocess with a Pipeline + ColumnTransformer

This handles numeric and categorical columns differently and prevents leakage automatically:

```

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression

num_cols = X.select_dtypes(include="number").columns
cat_cols = X.select_dtypes(exclude="number").columns

preprocess = ColumnTransformer([
    ("num", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ]), num_cols),
    ("cat", Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), cat_cols),
])

model = Pipeline([
    ("prep", preprocess),
    ("clf", LogisticRegression(max_iter=1000)),
])

```

5. Train and evaluate

```

from sklearn.metrics import classification_report

model.fit(X_train, y_train)
print(classification_report(y_test, model.predict(X_test)))

```

For a more reliable estimate, use cross-validation on the whole pipeline:

```

from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5)
print(scores.mean(), scores.std())

```

Common gotchas

- **ID columns:** drop them (`df.drop(columns=["id"])`) so the model doesn't learn from them.
- **Numbers stored as text:** if a numeric column shows as `object` in `df.dtypes`, fix it

```
with pd.to_numeric(df[col], errors="coerce") .
```

- **Regression instead of classification:** swap in e.g. `LinearRegression` or `RandomForestRegressor` and drop `stratify=y`.
- **Tree models** (random forests, gradient boosting) don't need scaling, but the pipeline still works fine with it.

References: [scikit-learn: Column Transformer with Mixed Types](#), [scikit-learn: Common pitfalls \(data leakage\)](#), [scikit-learn: Pipelines and composite estimators](#).

Team code modifications:

(to be filled in)

Turn 3

Prompt: (accidental keystrokes — no substantive content)

```
mo0 p]} |<<<<<<}}}}}}}}}}}}}}}}}}}}}}}}}}}}}}}}}}
```

Raw AI output:

Looks like that one might have been a stray keypress (or a cat on the keyboard). What would you like to do next? For example, I can help with a specific dataset, choosing a model, or tuning hyperparameters.

Team code modifications: n/a

Turn 4

Prompt:

so this is all from the examples section, what is the difference between a column transformer vs a pipeline

Raw AI output:

They solve different problems: a **Pipeline chains steps in sequence**, while a **ColumnTransformer runs different steps side by side on different columns**.

Pipeline: sequential, same data flows through every step

```
Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
])
```

Data goes in, gets imputed, and that output gets scaled. Every step sees *all* the columns it receives. All steps except the last must be transformers (have `fit / transform`); the last can be a model (`fit / predict`).

`X → impute → scale → (model)`

ColumnTransformer: parallel, each branch gets a subset of columns

```
ColumnTransformer([
    ("num", numeric_steps, ["age", "income"]),
    ("cat", categorical_steps, ["city", "gender"]),
])
```

It splits the columns, sends each group to its own transformer, then concatenates the results horizontally into one feature matrix. Columns you don't list are dropped by default (use `remainder="passthrough"` to keep them).

```

      ↗ [age, income] → numeric_steps ↖
X ———|                                     |→ combined matrix
      ↘ [city, gender] → categorical_steps ↙
```

Why the example nests them

Real data has mixed types, and you can't scale text or one-hot encode continuous numbers. So the example uses:

1. Small **Pipelines** inside each branch (impute *then* scale; impute *then* encode), because those steps must happen in order.
2. A **ColumnTransformer** to route numeric vs. categorical columns to the right mini-pipeline.
3. An outer **Pipeline** to chain the whole preprocessing block to the model.

```
Pipeline
├── ColumnTransformer
│   ├── Pipeline(impute → scale)           on numeric columns
```

| └─ Pipeline(impute → one-hot) on categorical columns
└─ LogisticRegression

Quick rule of thumb

Question	Use
"Do X, then Y, then Z"	Pipeline
"Do X to these columns and Y to those columns"	ColumnTransformer

The payoff of wrapping everything in the outer Pipeline is that `model.fit(X_train, y_train)` fits imputers, scalers, and encoders only on training data, and `model.predict(X_test)` reapplies those same learned transformations. That's what prevents leakage and also lets you pass the whole thing to `cross_val_score` or `GridSearchCV` as a single object.

References: [scikit-learn user guide: Pipelines and composite estimators](#) (sections 6.1.1 Pipeline and 6.1.4 ColumnTransformer), [ColumnTransformer API](#), [Pipeline API](#).

Team code modifications:

(to be filled in)

Turn 5

Prompt:

how am i able to tell the difference between categorical and numerical?

Raw AI output:

The dtype gives you a first guess, but the real test is about **meaning**: does arithmetic on the values make sense?

The core question

- **Numerical**: the numbers have magnitude and distance. 40 years old is twice 20, and the gap from 20 to 30 equals 30 to 40. Examples: age, price, temperature, count.
- **Categorical**: the values are labels. Averaging them is meaningless. Examples: city, color, product type, and also *numbers used as codes*, like zip codes. Zip 75050 isn't "more" than 76010, so the mean zip code is nonsense.

Step 1: Check what pandas inferred

```
df.dtypes
```

- `int64`, `float64` → pandas thinks numeric
- `object`, `string`, `category`, `bool` → pandas thinks categorical (or text)

That's what `select_dtypes(include="number")` in the example relies on, which is why it's only a starting point.

Step 2: Look at the number of unique values

```
df.nunique().sort_values()
```

- A numeric column with only a handful of unique values (e.g. 0/1, or 1–5) may really be categorical.
- An `object` column where nearly every row is unique is probably an ID, name, or free text, and usually should be dropped rather than one-hot encoded.

Step 3: Eyeball the actual values

```
df["col"].value_counts().head(20)
df.describe()                    # numeric summary
df.describe(include="object")    # categorical summary
```

Cases where the dtype lies

Looks like	Actually	Example	Fix
Numeric	Categorical	Zip code, store ID, month as 1–12	<code>df[col] = df[col].astype("category")</code>
Categorical	Numeric	"\$1,200", "45%", or a column with a stray "N/A"	Clean text, then <code>pd.to_numeric(df[col], errors="coerce")</code>
Numeric	Ordinal	Rating 1–5, education level	Judgment call (see below)

Ordinal: the in-between case

Ordinal variables have an order but uneven or undefined spacing ("low < medium < high", survey ratings). Options:

- Treat as numeric if the order matters and the spacing is roughly even (a 1–5 rating is often fine this way).
- Use `OrdinalEncoder` with an explicit order for text labels:

```
OrdinalEncoder(categories=[["low", "medium", "high"]])
```

- One-hot encode if you don't want the model assuming any spacing.

A practical pattern

Rather than trusting `select_dtypes` blindly, list the columns yourself once you've inspected them:

```
num_cols = ["age", "income", "tenure_months"]
cat_cols = ["city", "plan_type", "zip_code"]    # zip_code overridden on purpose
```

Explicit lists are easier to reason about and won't silently change if a column's dtype shifts after cleaning.

References: [pandas: Categorical data](#), [scikit-learn: Encoding categorical features](#) (covers `OrdinalEncoder` vs. `OneHotEncoder`), [pandas to_numeric](#) .

Team code modifications:

(to be filled in)

Turn 6

Prompt:

does this have anything to do with stevens ladder?

Raw AI output:

Yes, directly. What I described is essentially a simplified version of Stevens' levels of measurement, from S. S. Stevens' 1946 paper in *Science*. His four levels form a ladder where each rung adds a property and allows more operations:

	What it		Meaningful	Typical ML
--	---------	--	------------	------------

Level	adds	Example	operations	handling
Nominal	Labels only	City, color, zip code	Equality, counts, mode	<code>OneHotEncoder</code>
Ordinal	Order	Low/med/high, 1–5 rating	Above, plus ranking, median	<code>OrdinalEncoder</code> or treat as numeric
Interval	Equal spacing	Temperature in °F/°C, calendar year	Above, plus addition/subtraction, mean	Numeric (scale it)
Ratio	True zero	Age, price, weight, Kelvin	Above, plus multiplication/ratios	Numeric (scale it)

So in scikit-learn terms:

- "Categorical" $\approx$ **nominal** (and sometimes ordinal)
- "Numerical" $\approx$ **interval + ratio**
- The ordinal case I called "in-between" is exactly the rung where the practical choice gets fuzzy

The zip code example is a nominal variable that happens to be written with digits. Stevens' framework is what explains why computing its mean is meaningless: averaging requires at least interval scale.

The interval vs. ratio distinction rarely changes preprocessing, since both get scaled the same way. It mainly matters for interpretation: 40°F isn't "twice as warm" as 20°F, but \$40 is twice \$20.

One caveat worth knowing: Stevens' rules about "permissible statistics" have been criticized as too rigid for real data analysis. Velleman and Wilkinson (1993) argued that a variable's type depends on the question you're asking, not a fixed property of the data. Treating a 1–5 rating as numeric, for instance, often works fine in practice even though it's strictly ordinal. So use the ladder as a guide for thinking, not a hard rule.

References:

- Stevens, S. S. (1946). "On the Theory of Scales of Measurement." *Science*, 103(2684), 677–680.
- Velleman, P. F., & Wilkinson, L. (1993). "Nominal, Ordinal, Interval, and Ratio Typologies Are Misleading." *The American Statistician*, 47(1), 65–72.

- [scikit-learn: Encoding categorical features](#)

Team code modifications: n/a (conceptual)

Turn 7

Prompt:

can you expand on steps 2 and 3. Im trying to understand what is meant by "seperate features and target". the split before processing makes sense, is the test size a percentage of total size?

Raw AI output:

Step 2: Features vs. target

In supervised learning, you're trying to **predict one column from the others**.

- **Target (y)**: the column you want to predict. Also called the label or response.
- **Features (x)**: all the other columns the model uses to make that prediction. Also called predictors or inputs.

Say you have this CSV and want to predict whether a customer cancels:

age	plan_type	monthly_bill	tenure_months	churned
34	basic	45.00	12	0
51	premium	89.99	48	0
23	basic	45.00	2	1

Here `churned` is the target and the other four columns are features.

```
X = df.drop(columns=["churned"])    # everything except the target
y = df["churned"]                  # just the target
```

- `X` is a DataFrame (2D: rows $\times$ feature columns)
- `y` is a Series (1D: one value per row)

The split exists because scikit-learn's API expects them separately: `model.fit(X, y)` means "learn how `X` relates to `y`," and `model.predict(X_new)` returns predicted `y` values. If

you left the target inside `x`, the model would just read the answer off that column, and you'd get near-perfect but meaningless results. That's a form of leakage too.

Which column is the target isn't something the data tells you; it comes from your problem. The same dataset could have `monthly_bill` as the target if you were instead predicting spending.

Step 3: `test_size` and the other arguments

```
train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

`test_size` : yes, when it's a float between 0 and 1, it's the **fraction of rows** held out for testing. With 1,000 rows, `test_size=0.2` gives 200 test rows and 800 training rows. It also accepts:

- An **integer**, meaning an absolute row count: `test_size=150` → exactly 150 test rows
- Nothing, in which case it defaults to 0.25

Common choices are 0.2 or 0.25. With small datasets, you're trading off: a bigger test set gives a more reliable score, but leaves less data to learn from. That tension is part of why cross-validation exists.

`random_state` : the split is random (rows are shuffled first). Setting a number makes it reproducible, so you get the same split every run. The value 42 has no significance; any integer works.

`stratify=y` : keeps the class proportions the same in both sets. If 10% of customers churned overall, stratifying guarantees roughly 10% in train and 10% in test. Without it, a random split on imbalanced data could put too few positive cases in the test set, which makes your evaluation unreliable. Use it for classification; leave it out for regression (continuous targets can't be stratified this way).

What you get back: four objects, always in this order:

```
X_train, X_test, y_train, y_test = train_test_split(...)
```

The rows stay aligned, so `y_train[i]` is still the target for `X_train`'s row `i`.

References: [train_test_split API docs](#) (defines the float/int/default behavior of `test_size`), [scikit-learn glossary: X and y](#), [scikit-learn: Cross-validation](#).

Team code modifications:

(to be filled in)

Turn 8

Prompt: (project specification `project1_specification.docx` attached)

This is the document that is used for a school project. Do not outright give me answers, I need you too treat this as a refrence document and help me understand it in the framework prodvided by skicit-learn docs. Do not give me any code that is specific to this document only give me code found in skikit-learn documents. Ensure everything you give to me has references so i can double check answers.

Raw AI output:

Here's the project mapped onto the scikit-learn framework. For each part you get the concept, the relevant official doc, code taken from those docs (not written for your dataset), and questions to work through yourself.

First, an integrity note. Section 5 of your spec says you must log every prompt you send to an AI tool, including the prompt text, model version, and raw output. That includes this conversation. For your appendix: this is Claude Opus 5, and you'll want to record your prompts and my responses.

Big picture: why the spec insists on Pipelines

The spec's central concern is data leakage: statistics like means, medians, bin edges, and category lists must be learned only from training data. The Pipeline + ColumnTransformer structure from your earlier question is how scikit-learn enforces that. The official "Column Transformer with Mixed Types" example is the closest match to what the spec asks for. It shows how to apply different preprocessing pipelines to different subsets of features, which is especially useful for heterogeneous data where you want to scale numeric features and one-hot encode categorical ones.

The core of that example, verbatim from the docs:

```
numeric_features = ["age", "fare"]
numeric_transformer = Pipeline(
    steps=[("imputer", SimpleImputer(strategy="median")), ("scaler", StandardScal
)]

categorical_features = ["embarked", "sex", "pclass"]
categorical_transformer = Pipeline(
```

```

steps=[
    ("encoder", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
    ("selector", SelectPercentile(chi2, percentile=50)),
]
)
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features),
    ]
)

```

Reference: [Column Transformer with Mixed Types](#)

Your job is to figure out what's different about your dataset and your spec compared to this example (hint: compare the encoder settings in this snippet against Task 4).

Task 1: SimpleImputer

What the docs say. The `strategy` parameter controls how missing values are filled. "median" replaces missing values with each column's median and only works on numeric data; "most_frequent" uses the most common value and works on strings or numbers; "constant" replaces missing values with `fill_value`. For string or object columns, `fill_value` must be a string.

There's also a parameter the spec doesn't mention but that's worth knowing for Challenge A: setting `add_indicator=True` stacks a MissingIndicator onto the imputer's output, which lets a model account for missingness even after imputation.

Questions to work through:

- Run `df.isna().sum()` on the raw data. Which columns actually have missing values? Do Quantity and UnitPrice have any? If not, what is the median imputer protecting against?
- The spec gives two options for categorical imputation (mode or a constant like 'Unknown'). Which is more appropriate for a column where 25% of values are missing? Think about what mode imputation would do to the distribution of that column.

Reference: [SimpleImputer API](#), [Imputation user guide](#)

Task 2: Scaling comparison

The official "Compare the effect of different scalers on data with outliers" example is

essentially the experiment this task asks for. Key points from it:

- StandardScaler removes the mean and scales to unit variance, but outliers influence the empirical mean and standard deviation it computes. As a result, StandardScaler cannot guarantee balanced feature scales when outliers are present.
- MinMaxScaler maps values into [0, 1], but in that example it compressed all the inliers into a narrow range of roughly [0, 0.005], and both StandardScaler and MinMaxScaler are very sensitive to outliers.
- RobustScaler's centering and scaling statistics are based on percentiles, so a few very large outliers don't influence them.

The docs' comparison code, verbatim:

```
distributions = [  
    ("Unscaled data", X),  
    ("Data after standard scaling", StandardScaler().fit_transform(X)),  
    ("Data after min-max scaling", MinMaxScaler().fit_transform(X)),  
    ("Data after max-abs scaling", MaxAbsScaler().fit_transform(X)),  
    (  
        "Data after robust scaling",  
        RobustScaler(quantile_range=(25, 75)).fit_transform(X),  
    ),  
],
```

Note: that example uses `fit_transform` on the whole dataset because it's a visualization, not a modeling workflow. In your project, the scaler lives inside the Pipeline so it only fits on training data.

Questions to work through:

- Your spec says Quantity ranges from -80,995 to +80,995. What does a single row with that value do to the standard deviation, and therefore to everyone else's z-score?
- Should cancellations (negative quantities) be scaled together with purchases, or handled first? The spec hints at this in Section 2.

Reference: [Compare the effect of different scalers on data with outliers](#), [Preprocessing user guide: Standardization](#)

Task 3: KBinsDiscretizer

The three strategies your spec names map directly to the docs:

- 'uniform': bin widths are constant in each feature.

- 'quantile': each bin has approximately the same number of samples.
- 'kmeans': bins are based on the centroids of a KMeans clustering.

The docs example loops over all three (verbatim):

```
strategies = ["uniform", "quantile", "kmeans"]
...
for strategy in strategies:
    enc = KBinsDiscretizer(
        n_bins=4,
        encode="ordinal",
        quantile_method="averaged_inverted_cdf",
        strategy=strategy,
    )
    enc.fit(X)
    grid_encoded = enc.transform(grid)
```

The `quantile_method` argument is from recent versions; check which scikit-learn version your Anaconda environment has, since the API has changed over time. Also note

`encode="ordinal"` : your spec says "discrete ordinal bins," so look up what the default `encode` value is and why that matters.

Questions to work through:

- UnitPrice is heavily right-skewed. With equal-width bins over a range up to £38,970, roughly how many transactions would land in the first bin?
- This connects to Stevens: what level of measurement does the output of KBinsDiscretizer have, and what level did the input have? What information is lost?

Reference: [Demonstrating the different strategies of KBinsDiscretizer](#), [KBinsDiscretizer API](#)

Task 4: OneHotEncoder

The two parameters your spec requires:

- `handle_unknown='ignore'` : when an unknown category appears during transform, its one-hot columns are all zeros. The default is 'error', which raises instead.
- `sparse_output=True` (the default) returns a SciPy sparse matrix in CSR format. This parameter was renamed from `sparse` in version 1.2.

That rename matters for your AI critique: code generated by an AI trained on older examples may use `sparse=` instead of `sparse_output=`.

Questions to work through:

- Look at the `nunique()` of Description, StockCode, and Country. What would one-hot encoding each of them produce in terms of column count?
- Why would 'Book', 'book', and 'BOOK' be a problem for OneHotEncoder specifically? Where in the workflow should text standardization happen, and does it risk leakage? (Think about whether lowercasing requires learning anything from the data.)
- Which columns are nominal even though they contain digits? (Same idea as the zip code case from earlier. Your rubric explicitly mentions "measurement scale theory.")

Reference: [OneHotEncoder API](#), [Encoding categorical features](#)

Task 5: Composition and the sparse-output question

Two ColumnTransformer details that matter for your AI critique:

- `sparse_threshold` (default 0.3): if the transformers produce sparse output, the combined result is stacked as sparse only if its overall density is below this value. Setting it to 0 always returns dense.
- By default, only the specified columns are transformed and all other columns are dropped; `remainder='passthrough'` keeps them.

And one StandardScaler detail: `with_mean=True` doesn't work on sparse matrices and raises an exception, because centering would require building a dense matrix that's likely too large for memory.

Questions to work through:

- The spec says the final output should be "a clean, model-ready NumPy matrix," but Task 4 requires sparse output. Is your final output actually a NumPy array or a sparse matrix? How would you check?
- If one of your pipelines scales a sparse matrix, which StandardScaler parameter would you need to change?

Reference: [ColumnTransformer API](#), [Pipelines and composite estimators](#), [Common pitfalls: data leakage](#)

Part I challenges: where to look (not answers)

Challenge A (MCAR / MAR / MNAR). scikit-learn's docs cover the mechanics of imputation

but not the theory of missingness mechanisms. The standard reference is Little & Rubin, *Statistical Analysis with Missing Data* (Wiley, 3rd ed. 2019), Chapter 1. Guiding question: in the scenario, missingness depends on whether the customer was a guest. Is "guest" observable anywhere else in the data? That decides which of the three mechanisms applies. Then think about what a single constant like 'Guest_999' does when a model aggregates "per customer." The `add_indicator` parameter above is scikit-learn's practical tool for informative missingness.

Challenge B (distance distortion). See the official [Importance of Feature Scaling](#) example, which demonstrates this with a KNN classifier. Guiding question: write out the Euclidean formula for two points that differ by £10 in price and 50 units in quantity. What fraction of the squared sum comes from each term? Then use the scalers example above for the outlier part.

Challenge C (SMC vs. Jaccard). scikit-learn's `DistanceMetric` docs define both in terms of counts. They use NTT for dimensions where both values are True, NTF and NFT for mismatches, NFF for dimensions where both are False, $NNEQ = NTF + NFT$, and $NNZ = NTF + NFT + NTT$. The "jaccard" distance is $NNEQ / NNZ$, while "matching" distance is $NNEQ / N$. Guiding question: SMC similarity is 1 minus the matching distance. Write both similarities in terms of NTT, NTF, NFT, NFF, then ask which term appears in one formula and not the other, and how large that term is when $N = 50,000$. Your course textbook likely uses f_{11}/f_{00} notation for the same counts (Tan, Steinbach & Kumar, *Introduction to Data Mining*, Ch. 2, is the usual source).

Reference: [DistanceMetric API](#)

If you want to go deeper on any one task, pick it and I can walk through the relevant doc section in more detail, or you can draft your reasoning for a challenge and I'll check it against the references.

Team code modifications:

(to be filled in)

Turn 9

Prompt:

another thing, do i still need to separate into features and target? How do i choose a target? everything else seems like scaffolding and then just choosing options that align with the project

Raw AI output:

For this project, you mostly don't need a target. Read Task 5 closely: the spec asks for `pipeline.fit(X_train).transform(X_test)` producing a "model-ready" matrix. There's no `y` in that line and no model at the end. Project 1 is about building the preprocessing stage, not training a predictor.

Why the pipeline works without a target

Every transformer in your spec is **unsupervised**: it learns its statistics from `x` alone. In the scikit-learn API, their `fit` signature is `fit(X, y=None)`, and the docs describe the `y` parameter as ignored and present only for API consistency. For example:

- `SimpleImputer` learns medians/modes from `X`
- `StandardScaler` learns means and standard deviations from `X`
- `KBinsDiscretizer` learns bin edges from `X`
- `OneHotEncoder` learns the category list from `X`

A Pipeline whose steps are all transformers can likewise be fit with `pipeline.fit(X_train)` alone.

References: [StandardScaler.fit](#), [Pipeline.fit](#), [Glossary: unsupervised / transformer](#)

What you still need: a train/test split of rows

Even without a target, you still split rows so the transformers fit on training data only.

`train_test_split` accepts any number of arrays, including just one. From its docs, the signature is `train_test_split(*arrays, test_size=None, ...)`, so a single `DataFrame` works and you get back two pieces instead of four.

Reference: [train_test_split API](#)

One thing to watch: the docs example from last time includes `SelectPercentile(chi2, ...)`. That step is supervised (chi2 scores features against `y`), so it won't work in a target-free pipeline. That's the kind of difference your AI critique section should catch.

If you did want a target

The challenges mention hypothetical models (CLV prediction in A, 3-NN buyer classification in B), but those are thought experiments for the report, not required implementations. If you're unsure whether your instructor expects a model, that's worth a quick question to her or the TA rather than a guess. If you ever do choose one, the rules are:

- The target comes from the question you're answering, not from the data.
- The target must not also appear in x , and neither should anything computed from it (for example, if you predicted revenue, `Quantity × UnitPrice` in the features would leak the answer).

On "the rest is scaffolding"

Partly true: the code structure is mostly standard. But look at the rubric. The code is 40%; the report and live demo are 50% combined, and the demo requires you to "defend architectural choices" and "test edge cases." So the real work is justifying each option. Two design questions the spec doesn't spell out, which are worth thinking through for that defense:

- **Splitting by invoice.** Each row is one line item, and one invoice has many rows. With a random split, lines from the same invoice land in both train and test. Is that a problem? Look at [GroupShuffleSplit](#) and the [cross-validation guide's section on grouped data](#) and decide.
- **Splitting by time.** The data spans Dec 2010 to Dec 2011. If this pipeline were used in production, it would transform *future* transactions. Does a random shuffle reflect that? See [TimeSeriesSplit](#).

Neither has a single right answer, which is exactly why they make good demo material.

Team code modifications:

(to be filled in)

Turn 10

Prompt:

how are you able to test for data leakage and what is the estimated output of something like this?

Raw AI output:

There are two ways to test for leakage: **structural checks** (prove each transformer learned its statistics only from training data) and a **behavioral test** (the experiment scikit-learn's own docs use). For a preprocessing-only project like yours, the structural checks are the main tool.

1. The principle you're testing

The docs give the rule directly: never include test data when using the `fit` and `fit_transform` methods, because using all the data, e.g. `fit(X)`, can result in overly optimistic scores. Conversely, `transform` should be used on both train and test subsets, since the same preprocessing should be applied to all the data. They also stress this isn't specific to one transformer: the risk of leakage is relevant with almost all transformations in scikit-learn, including `StandardScaler`, `SimpleImputer`, and `PCA`.

So a leakage test boils down to: **are the learned values the training set's values, and are they applied unchanged to the test set?**

2. What leakage-free behavior looks like (docs examples with their documented outputs)

These are from the "Examples" section of each API page, so you can verify the outputs yourself.

StandardScaler: new data is transformed with the *fitted* mean, not its own.

```
>>> from sklearn.preprocessing import StandardScaler
>>> data = [[0, 0], [0, 0], [1, 1], [1, 1]]
>>> scaler = StandardScaler()
>>> print(scaler.fit(data))
StandardScaler()
>>> print(scaler.mean_)
[0.5 0.5]
>>> print(scaler.transform(data))
[[-1. -1.]
 [-1. -1.]
 [ 1.  1.]
 [ 1.  1.]]
>>> print(scaler.transform([[2, 2]]))
[[3. 3.]]
```

The point `[2, 2]` becomes `3.0` because it's measured against the training mean (0.5) and std (0.5). If the scaler had refit on it, the result would be meaningless for a single point.

SimpleImputer: missing values are filled with the statistic learned during `fit`, not from the data being transformed.

```
>>> import numpy as np
>>> from sklearn.impute import SimpleImputer
>>> imp_mean = SimpleImputer(missing_values=np.nan, strategy='mean')
>>> imp_mean.fit([[7, 2, 3], [4, np.nan, 6], [10, 5, 9]])
SimpleImputer()
>>> X = [[np.nan, 2, 3], [4, np.nan, 6], [10, np.nan, 9]]
```

```
>>> print(imp_mean.transform(X))
[[ 7.    2.    3. ]
 [ 4.    3.5  6. ]
 [10.    3.5  9. ]]
```

The 3.5 is the mean of 2 and 5 from the *fit* data. The transform data's own column-2 mean would be 2.0 . That difference is exactly what you'd look for.

OneHotEncoder: categories come from fit only; unseen ones become all-zero rows.

```
>>> enc = OneHotEncoder(handle_unknown='ignore')
>>> X = [['Male', 1], ['Female', 3], ['Female', 2]]
>>> enc.fit(X)
OneHotEncoder(handle_unknown='ignore')
>>> enc.categories_
[array(['Female', 'Male'], dtype=object), array([1, 2, 3], dtype=object)]
>>> enc.transform([['Female', 1], ['Male', 4]]).toarray()
array([[1., 0., 1., 0., 0.],
       [0., 1., 0., 0., 0.]])
```

The value 4 wasn't seen during fit, so its three columns are all zeros instead of raising an error.

References: [StandardScaler](#), [SimpleImputer](#), [OneHotEncoder](#)

3. Applying this to your own pipeline

Each fitted transformer stores what it learned in attributes ending in an underscore. Those are your evidence:

Transformer	Fitted attribute to inspect
SimpleImputer	statistics_
StandardScaler	mean_ , scale_
KBinsDiscretizer	bin_edges_
OneHotEncoder	categories_

You reach them through the pipeline using `named_steps` (on a Pipeline) and `named_transformers_` (on a fitted ColumnTransformer); both are documented on their API pages. The checks, and what a leakage-free result looks like:

--	--	--

Check	Expected if no leakage	Red flag
Compare <code>statistics_ / mean_</code> to the same stat computed on the training split	Match	—
Compare them to the stat computed on the full dataset	Usually slightly different	Exact match on the full data (the fit saw test rows)
Mean / std of transformed train numeric columns	≈ 0 and ≈ 1	—
Mean / std of transformed test numeric columns	Close to 0 and 1, but not exact	Exactly 0 and 1 (the scaler was fit on test)
<code>categories_</code>	Only categories present in training	Categories that appear only in test
A test row with a category never seen in training	All-zero block for that feature	Error, or a matching column exists

The "not exact" row is the one people find counterintuitive: **imperfect test statistics are the evidence that you didn't leak**. I can't give you actual numbers, because they depend on your split and data, but that's the pattern to look for. These checks also make good material for the live demo's "test edge cases" requirement.

4. The behavioral test from the docs

The docs' leakage demonstration uses a clever setup: completely random features and random labels, so no real signal exists. Using all the data to perform feature selection results in an accuracy score much higher than chance, even though the targets are completely random. Done correctly inside a pipeline, the documented results land at chance level: an `accuracy_score` of 0.5 on the test split, and a mean cross-validated accuracy of 0.43+/-0.05.

So the "estimated output" for this kind of test is: **near 0.5 accuracy for a binary problem means no leakage; well above 0.5 on random labels means leakage**. This test needs a `y` and a model, so it's optional for your project, but it's a strong way to show you understand the concept.

Reference: [Common pitfalls and recommended practices, section 10.2](#)

5. What these tests can't catch

The checks above prove the pipeline doesn't leak *statistics*. They can't detect leakage built into the **split itself**, like line items from one invoice landing in both train and test, or future months informing past ones. Those have to be reasoned about, which ties back to the `GroupShuffleSplit` and `TimeSeriesSplit` questions from before.

Team code modifications:

(to be filled in)

Turn 11

Prompt:

Printing is not an option for this, the size is massive. I'm thinking. i could take a random subset of the data and use that so i can print.

this is my current output

```
<Compressed Sparse Row sparse matrix of dtype 'float64'  
with 864208 stored elements and shape (108382, 61595)>
```

Raw AI output:

Your instinct is right that you shouldn't print it, but a random subset isn't the best fix. That output already tells you a lot, and you can inspect it without converting anything to a dense array.

Reading the output you have

Why printing is impossible. The dense version would have $108,382 \times 61,595 \approx 6.68$ billion cells. At 8 bytes per float64, that's about 53 GB. The sparse version stores only the 864,208 nonzero values plus their positions, which is roughly 10 MB. This is exactly why the spec requires `sparse_output=True`.

Density. $864,208 / 6.68 \text{ billion} \approx 0.013\%$ of cells are nonzero.

Nonzeros per row. $864,208 / 108,382 \approx 8.0$. That's a useful diagnostic. Each one-hot encoded column contributes exactly one nonzero per row, and each numeric column contributes one value (unless it's exactly 0). So about 8 nonzeros per row means roughly 8 input columns went through the ColumnTransformer. Does that match what you intended?

Row count. 108,382 is 20% of 541,909, so this is your transformed test set. Good sign: you're transforming test, not fitting on it.

The column count is the thing to investigate. 61,595 output columns is far more than the number of distinct products in this dataset. Ask yourself: which of your input columns have tens of thousands of unique values, and are any of them going into the OneHotEncoder? Run `nunique()` on each input column and compare. Think back to the Stevens discussion: a column can be nominal and still be a poor feature. An identifier or a timestamp one-hot encoded gives the model a column per invoice or per second, which it can't generalize from.

Inspecting without printing

Instead of subsampling rows, **slice the output by columns** using attributes the fitted `ColumnTransformer` already provides:

- `output_indices_` is a dictionary mapping each transformer name to a slice of the transformed output, meant for inspecting which transformer produced which features. The width of each slice tells you exactly how many columns each branch created, which answers the question above immediately.
- `get_feature_names_out()` gives the name of every output column (for example `cat__Country_France`). Since `verbose_feature_names_out` defaults to `True`, each name is prefixed with the name of the transformer that generated it.

Using the `num` slice, you can pull just the numeric columns (a handful of columns instead of 61,595) and run the leakage checks from before (mean ≈ 0 , std ≈ 1 on train, close-but-not-exact on test). That's a small, printable result that uses all the rows.

SciPy sparse matrices also support row and column slicing and have `.nnz` and `.shape` directly, so you never need to densify the whole thing.

References: [ColumnTransformer API \(`output\_indices\_`, `get\_feature\_names\_out` \)](#), [SciPy `csc\_matrix` docs](#)

If you still want a subset

Subsampling is reasonable for **speeding up development** (iterating on the pipeline while you build it), with two cautions:

1. **Your final notebook should run on the full data.** Fitting on a subset changes the learned medians, means, bin edges, and category lists, so the results you report would differ from the real pipeline.
2. **Subsample before splitting, then split the subset.** If you subsample the test set after fitting and only look at it, that's fine. If you subsample and *refit*, make sure the refit sees only training rows.

If the column count turns out to be a problem

After you've identified which columns are causing the blowup, OneHotEncoder has built-in options worth reading about. `min_frequency` sets a threshold below which a category is treated as infrequent, and `max_categories` sets an upper limit on output features per input feature, with infrequent categories grouped into one. Whether you use those or drop a column entirely is a design decision you'd defend in the demo.

Reference: [OneHotEncoder: infrequent categories](#)

Team code modifications:

(to be filled in)

Turn 12

Prompt:

okay i am only working on the categoriacal portion and my teammate is working on the numerical portion

Raw AI output:

That split lines up well with how the ColumnTransformer works, since each branch is independent. Here's what matters for your side, plus the few places where you and your teammate have to coordinate.

Coordination points (agree on these early)

1. Column assignment. Each of you owns one list of columns in the ColumnTransformer. The lists should not overlap, and together they should cover every column you intend to use. Anything not listed is silently dropped: by default, only the specified columns are transformed and the rest are dropped unless you set `remainder='passthrough'`. So a column neither of you claims just disappears without an error. Decide together which columns are numeric, which are categorical, and which are intentionally dropped.

2. Final output format. Your branch produces sparse output; your teammate's likely produces dense. The ColumnTransformer decides what the combined result is: if any transformer output is sparse, the result is stacked as sparse only if its overall density is below `sparse_threshold` (default 0.3). With density around 0.013%, your combined output will be sparse. That affects the spec's "NumPy matrix" wording and your AI critique, so agree on how you'll describe it.

3. Branch names. The names you give each branch (like `"num"` and `"cat"`) become the

keys in `output_indices_` and the prefixes in `get_feature_names_out()`. Agree on them so your notebook sections and demo line up.

4. The demo. The rubric requires all three members to "demonstrate technical mastery" and defend architectural choices. Each of you should be able to explain the other's branch, not only your own.

Your categorical branch: decisions to work through

These are the choices you'll need to justify. I'll frame them as questions, per your earlier request.

Which columns are categorical at all? The 61,595-column output means something high-cardinality is going through the encoder. Run `nunique()` on every column you're sending to your branch. For each one, ask: is this a nominal attribute a model could learn from, or an identifier? The rubric explicitly grades "measurement scale theory," so this is where the Stevens framework earns points.

Where does text cleaning happen? Task 4 notes that Description has mixed casing and punctuation. Ask whether lowercasing or stripping punctuation *learns* anything from the data. If it doesn't (it's stateless), doing it before the split can't leak information. If you want it inside the pipeline anyway for a cleaner design, look at [FunctionTransformer](#), which wraps an arbitrary function as a pipeline step. Its docs example uses `np.log1p`, but the idea applies to any stateless function.

Imputation strategy. The spec allows `most_frequent` or a constant like `'Unknown'`. For each categorical column with missing values, ask: if 25% of values are missing and you fill them with the mode, what happens to that category's count? What does a constant preserve that the mode doesn't? This connects directly to Challenge A.

Encoder configuration. Two settings are required by the spec: `handle_unknown='ignore'` (unknown categories produce all-zero columns) and `sparse_output=True` (the default, returning a CSR matrix). Beyond that, if a column is legitimately categorical but has thousands of rare values, `min_frequency` and `max_categories` let you group infrequent categories together.

How to test your branch in isolation

Using the fitted ColumnTransformer:

- The width of `output_indices_['cat']` (or whatever you name it) tells you exactly how many columns your branch produced. That's your number to drive down if it's still in the tens of thousands.

- `get_feature_names_out()` filtered to your prefix shows which categories created those columns.
- Your fitted encoder's `categories_` should contain only values seen in training.
- Feed it a row with a category that never appeared in training and confirm you get an all-zero block rather than an error. That doubles as a live-demo edge case.

References: [ColumnTransformer API](#), [OneHotEncoder API](#), [Encoding categorical features](#), [SimpleImputer API](#)

A good next step is the `nunique()` check. Once you know which columns are driving the column count, you can make the drop/keep/group decisions.

Team code modifications:

(to be filled in)

Turn 13

Prompt:

see there are quite a few nominal items that i have in my categorical section. too me it seams like of all of them. the issue im having is since we don't have anything we are aiming for besides creating a a preprocessing pipline and setting up this sparse matrix. I beleive i cannot condense or derive anything.

Raw AI output:

You're right about one part: without a target, you can't use **supervised** reduction. Anything that scores categories against `y` is off the table, such as `TargetEncoder` (it encodes categories using the target's mean) or the `SelectPercentile(chi2)` step in the docs example.

But "no target" doesn't mean "no condensing or deriving." Several of the most important decisions here don't depend on a target at all.

1. Frequency-based grouping is unsupervised

`min_frequency` and `max_categories` on `OneHotEncoder` decide which categories are "infrequent" by counting how often each appears in the **training data**. No `y` is involved; it's learned from `x_train` during `fit` like any other statistic, so it's leakage-safe inside the pipeline.

Reference: [Infrequent categories](#)

2. "Nominal" and "useful feature" aren't the same thing

Every identifier is nominal, but not every nominal column belongs in the encoder. Your spec asks for a "model-ready" matrix, and whether something is model-ready doesn't require choosing a model. You can test it on the data itself:

- For each high-cardinality column, what fraction of **test** values also appear in **train**? With `handle_unknown='ignore'`, every unseen value becomes an all-zero row. If most test values for a column are unseen, what information is that column actually carrying into the test set?
- If an identifier's values *do* mostly appear in both train and test, why? (Look back at the invoice-grouping question from earlier. Those two issues are connected.)

Dropping or keeping a column is a legitimate preprocessing decision, and "this is an identifier, not an attribute" is a measurement-scale argument your rubric explicitly rewards.

3. Deriving features can be stateless

Some derivations don't learn anything from the data, so they can't leak and don't need a target:

- **Dates:** a raw timestamp one-hot encoded is nearly all unique values. What happens if you extract components like month, weekday, or hour instead? scikit-learn has a full example on this: [Time-related feature engineering](#).
- **Codes with embedded meaning:** read the variable descriptions on the [UCI Online Retail dataset page](#). One of the identifier columns has a documented pattern that encodes something your spec explicitly mentions in Section 2. Would extracting that be more useful than encoding the raw ID?
- **Text:** the case/punctuation cleanup from Task 4 is itself a derivation that condenses Description.

A stateless function can live inside the pipeline via [FunctionTransformer](#).

A way to frame it for the report and demo

For each categorical column, you could document four things: its measurement level, its number of unique values in train, the fraction of test values unseen in train, and your decision (encode as-is, group infrequent, derive, or drop) with the reasoning. That turns "I believe I can't condense anything" into a defensible table, and it's the kind of architectural justification the 20% demo grade asks for.

Team code modifications:

(to be filled in)

Turn 14

Prompt:

do the numerical values need cleaning or only catogorical?

Raw AI output:

Both need cleaning, and the spec says so directly. Section 2's table flags two numeric problems alongside the categorical ones: Quantity ranging from -80,995 to +80,995 with negative values signifying cancellations, and UnitPrice reaching £38,970, with the note that extreme values severely distort standard scaling. So your teammate has real work, not just "drop in a StandardScaler."

The important structural point for the two of you:

Three tiers of cleaning

1. Row-level decisions — shared, and they happen before the ColumnTransformer.

Filtering rows affects every column, so these are joint decisions, not yours or your teammate's alone:

- Do cancellation rows (negative Quantity) stay? If they're removed, your categorical branch sees fewer rows too, and your `categories_` may change.
- What about rows with zero or negative UnitPrice, or missing Description?

A row-filtering rule with fixed thresholds you chose yourself is stateless, so it can't leak. A rule that computes a cutoff *from the data* (say, dropping above the 99.9th percentile) does learn something, and where you apply it matters. Worth thinking through which kind each of your rules is.

2. Type and parsing issues — whoever owns the column.

Check `df.dtypes` on the raw load. Ask whether every column you consider numeric actually parsed as `int64` / `float64`, and whether every column you consider categorical parsed as `object`. Any mismatch is a cleaning task. There's one identifier column in this dataset that commonly parses as a float because of its missing values, which is relevant to *your* branch, not your teammate's, if you're treating it as nominal.

3. Transformer-handled issues — each branch separately.

Imputation, scaling, and binning are all learned inside the pipeline, which is what the ColumnTransformer structure is for.

One thing to settle between you

The spec's Task 1 says to impute numeric columns with the median. Run `df.isna().sum()` and check whether the numeric columns actually contain any missing values in this dataset. If they don't, your teammate should be ready to explain in the demo why the imputer is in the pipeline anyway. (Hint: think about what happens at `transform` time on data the pipeline has never seen, which is the situation the spec's "unseen test folds" language keeps pointing at.)

References: [SimpleImputer API](#), [Compare the effect of different scalers on data with outliers](#), [Common pitfalls: data leakage](#), [UCI Online Retail dataset page](#) for the column descriptions.

Team code modifications:

(to be filled in)

Turn 15

Prompt:

can you output this into a jf/Ai_ChatLog markdown file

Raw AI output: this file.

Notes for the final report

- The spec's Section 5 also requires a **Required AI Evaluation Exercise**: running the provided production-grade-script prompt in an AI interface and including the exact unedited output, the team's finalized code, and a critique of whether the AI handled sparse output, set `handle_unknown='ignore'`, and prevented leakage. That exercise is **not** in this log and still needs to be run and recorded separately.
- No code in this log was written specifically for the UCI Online Retail dataset; all snippets are from the official scikit-learn documentation, which is BSD-licensed. Cite the doc pages listed in each turn.

- Teammates who used AI tools separately need to append their own prompt logs.